



# **SYMBIOSIS**

## **INSTITUTE OF TECHNOLOGY, NAGPUR**

### **Practical File Submission**

**Course Name: JavaScript Lab**

**Course Code: T-3013 / 0705210508**

**Submitted by**

Siddhant Virmani

24070521206

Batch: 2024-28

Sem- V

**Submitted to**

Dr. Shreyas Rajendra Hole

## Practical 1

**Experiment Title: Demonstrate inline, internal, and external JavaScript; use console methods; create a webpage showing user info and a welcome message.**

**Software/Tools Required:** VS Code , Web Browser

**Experiment Program Code:**

**Demo html file:**

```
<!DOCTYPE html>

<html>

<head>

  <title>CSS and JavaScript Demonstration</title>


  <!-- External CSS -->
  <link rel="stylesheet" href="external.css">


  <!-- Internal CSS -->
  <style>
    body{
      text-align:center;
      font-family:Arial, sans-serif;
    }

    h1{
      color:darkblue;
    }
```

```
.internal{  
    color:darkblue;  
}
```

```
button{  
    padding:10px 20px;  
    font-size:20px;  
    background:#1f4e79;  
    color:white;  
    border:none;  
    border-radius:5px;  
    cursor:pointer;  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h1 class="internal">Welcome to Internal CSS</h1>
```

```
<h2>Name: Om shendre</h2>
```

```
<h2>PRN: 24070421195</h2>
```

```
<!-- Inline CSS -->
```

```
<h1 style="color:green;">Welcome to Inline CSS</h1>
```

```
<h2>Name: Om shendre</h2>
```

```
<h2>PRN: 24070421195</h2>
```

<!-- External CSS -->

<h1 class="external">Welcome to External CSS</h1>

<h2>Name: Om shendre</h2>

<h2>PRN: 24070421195</h2>

<!-- Inline JavaScript -->

<button onclick="showMessage()">Click Me</button>

<h2 id="msg"></h2>

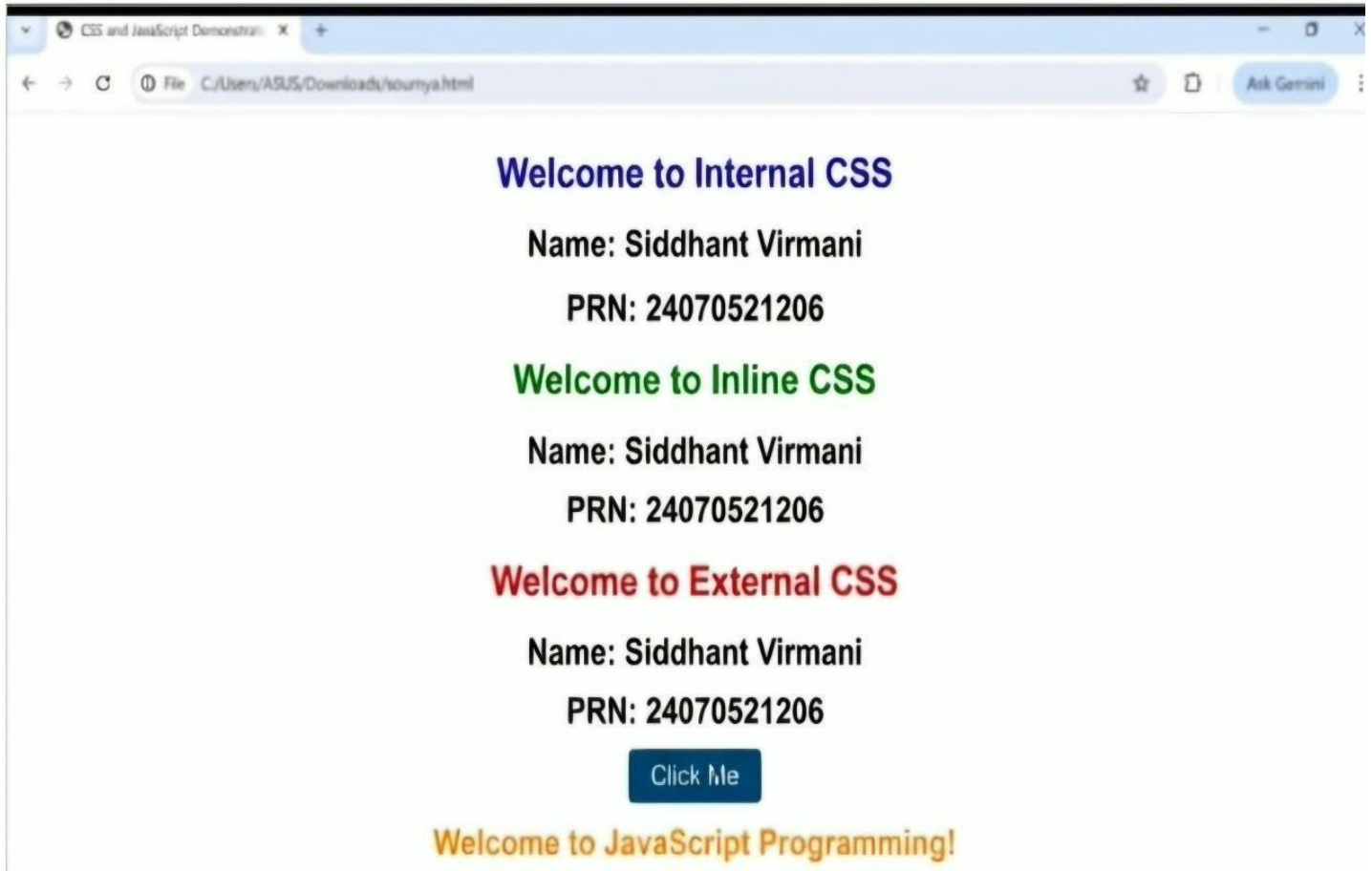
<!-- External JavaScript -->

<script src="script.js"></script>

</body>

</html>

**Output :**



## Case Study 01

**Case Study Title:-** Create a web page for college department and link the student information page with it and write a js program to display "Welcome to SIT Nagpur" in an alert box

**Case Study Code:**

**Department file (HTML):-**

```
<!DOCTYPE html>
<html>

<head>
  <title>SIT Nagpur Department</title>

  <style>
    body {
      background-color: rgb(176,213,216);
      font-family: Arial, sans-serif;

      display: flex;
      justify-content: center;
      align-items: center;
      height: 100vh;
      margin: 0;
    }

    .box {
      background-color: white;
      padding: 30px;
      border: 2px solid #000;
      border-radius: 10px;
```

```
text-align: center;
width: 400px;
}
</style>
</head>

<body>

<div class="box">
  <h1>SIT Nagpur - Computer Science Department</h1>

  <p>Welcome to our official SIT, Nagpur department page.</p>
  <p>Om Shendre</p>
  <p>PRN-24070521195</p>

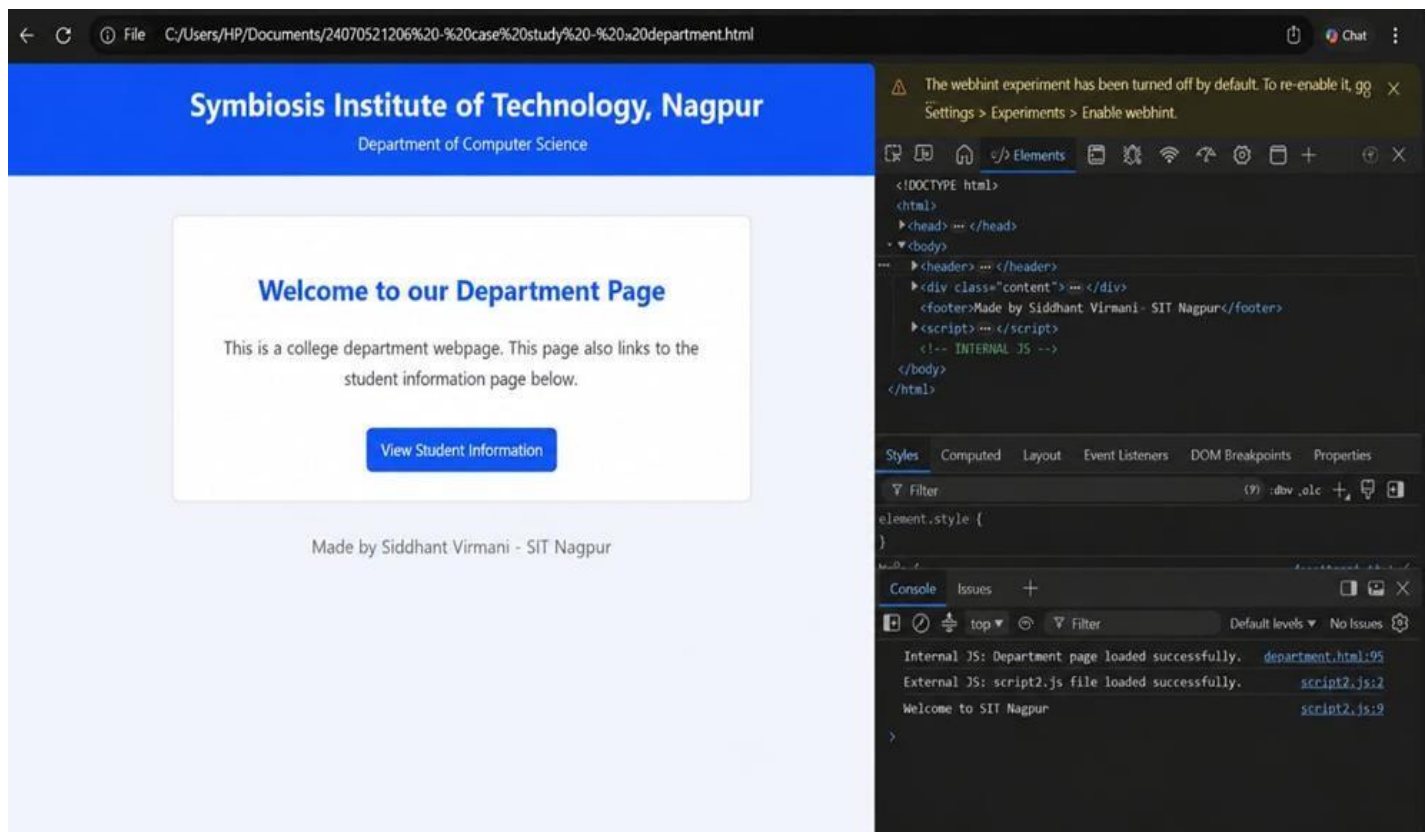
  <p>
    <a href="student-info.html">Click here to view Student Info</a>
  </p>
</div>

<script>
  alert("Welcome to SIT Nagpur");
</script>

</body>

</html>
```

**Output:**



## Result / Conclusion:-

Hence we have created a basic html page with javascript to show student information and website login page.

## Practical 02

**Aim:-** Use var, let, const, and demonstrate basic features like template literals and destructuring; build a billing calculator with user input.

**Software/Tools Required:** VS Code , Web Browser

**Experiment program code:**

**Calculator.html:-**

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Billing Calculator</title>
```

```
</head>
```

```
<body>
```

```
  <h2>Billing Calculator</h2>
```

```
  <script>
```

```
    const GST = 0.18;
```

```
    var customerName = prompt("Enter Customer Name: ");
```

```
    var productName = prompt("Enter Product Name: ");
```

```
    var quantity = Number(prompt("Enter quantity"));
```

```
    let price = Number(prompt("Enter price per Item"));
```

```
    let total = quantity * price;
```

```
    let gstAmount = total * GST;
```

```
let finalBill = total + gstAmount;
```

```
const bill = {  
  customer: customerName,  
  product: productName,  
  quantity,  
  price,  
  total,  
  gstAmount,  
  finalBill  
};
```

```
const {  
  customer,  
  product,  
  gstAmount: gst,  
  finalBill: amount  
} = bill;
```

```
document.write("<h3>Billing Receipt</h3>");  
document.write(`<p>Customer: ${customer}</p>`);  
document.write(`<p>Product: ${product}</p>`);  
document.write(`<p>Quantity: ${quantity}</p>`);  
document.write(`<p>Price per Item: ₹${price}</p>`);  
document.write(`<p>Total: ₹${total}</p>`);  
document.write(`<p>GST (18%): ₹${gst.toFixed(2)}</p>`);  
document.write(`<h3>Final Bill: ₹${amount.toFixed(2)}</h3>`);
```

```
</script>
```

```
</body>
```

```
</html>
```

**Output:-**



SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR  
Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)  
Re-accredited by NAAC with 'A++' Grade | Awarded Category - I by UGC  
Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Billing Calculator

File

C:/Users/sit.lab3/Desktop/javascript/Billing.html

## Billing Calculator

### Billing Receipt

Customer: Siddhant Virmani

Product: 24070521206

Quantity: 40

Price per Item: ₹20000

Total: ₹800000

GST (18%): ₹144000.00

**Final Bill: ₹944000.00**

## Case Study 02

**Case Study Title:-** Develop the webpage for an online shopping website with using HTML,CSS and HTML5 tags

**Software/Tools Used:-** VScode , Web Browser

**Code:-**

```
<!DOCTYPE html>

<html>

<head>

    <title>Online Shopping Store - Bill Calculator</title>

</head>


<body>


<h1>Online Shopping Store - Bill Calculator</h1>


<form id="billForm">


<label>Customer Name</label><br>

<input type="text" id="customer" required><br><br>


<label>Mobile Number</label><br>

<input type="text" id="mobile" required><br><br>


<label>Invoice Number</label><br>

<input type="text" id="invoice" required><br><br>


<label>Product Name</label><br>

<input type="text" id="product" required><br><br>
```



<label>Quantity (in Kg)</label><br>

<input type="number" id="qty" required><br><br>

<label>Rate per Kg (Rs.)</label><br>

<input type="number" id="rate" required><br><br>

<label>Discount (%)</label><br>

<input type="number" id="discount" value="0"><br><br>

<label>GST (%)</label><br>

<input type="number" id="gst" value="18"><br><br>

<label>Packing Charges (Rs.)</label><br>

<input type="number" id="packing" value="0"><br><br>

<label>Payment Mode</label><br>

<select id="payment">

<option>Cash</option>

<option>Card</option>

<option>UPI</option>

</select>

<br><br>

<label>Membership</label><br>

<select id="member">

<option>No</option>

<option>Yes</option>

</select>

<br><br>

<input type="button" value="Calculate Bill" onclick="calculateBill()">

</form>

<hr>

<div id="result"></div>

<script>

function calculateBill()

{

let customer=document.getElementById("customer").value;

let mobile=document.getElementById("mobile").value;

let invoice=document.getElementById("invoice").value;

let product=document.getElementById("product").value;

let qty=parseFloat(document.getElementById("qty").value);

let rate=parseFloat(document.getElementById("rate").value);

let discount=parseFloat(document.getElementById("discount").value);

let gst=parseFloat(document.getElementById("gst").value);

let packing=parseFloat(document.getElementById("packing").value);

let payment=document.getElementById("payment").value;

let member=document.getElementById("member").value;



```
let subtotal=qty*rate;

let discountAmount=subtotal*discount/100;

let afterDiscount=subtotal-discountAmount;

let gstAmount=afterDiscount*gst/100;

let membershipDiscount=0;

if(member=="Yes")
{
    membershipDiscount=afterDiscount*0.05;
}

let total=afterDiscount+gstAmount+packing-membershipDiscount;

document.getElementById("result").innerHTML=
`
<b>Customer:</b> ${customer} (${mobile})<br><br>

<b>Invoice No:</b> ${invoice}<br><br>

<b>Product:</b> ${product} (${qty} Kg)<br><br>

<b>Subtotal:</b> Rs. ${subtotal.toFixed(2)}<br><br>

<b>Discount:</b> Rs. ${discountAmount.toFixed(2)}<br><br>

<b>GST:</b> Rs. ${gstAmount.toFixed(2)}<br><br>
```

```
<b>Packing Charges:</b> Rs. ${packing.toFixed(2)}<br><br>

<b>Membership Discount:</b> Rs. ${membershipDiscount.toFixed(2)}<br><br>

<b>Payment Mode:</b> ${payment}<br><br>

<h3>Total Bill: Rs. ${total.toFixed(2)}</h3>

`;

}

</script>

</body>

</html>
```

**Output:-**



**SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR**  
**Symbiosis International (Deemed University)**

(Established under section 3 of the UGC Act, 1956)  
Re-accredited by NAAC with 'A++' Grade | Awarded Category - I by UGC  
Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

← ↻ ⓘ File C:/Users/HP/Documents/24070521191%20-%20js%20exp/exp%20/case-study-2.html 🔍 ⭐ 🗂 Chat

### Online Shopping Store - Bill Calculator

Customer Name

Mobile Number

Invoice Number

Product Name

Quantity (in Kg)

Rate per Kg (Rs.)

Discount (%)

GST (%)

Packing Charges (Rs.)

Payment Mode

Membership

---

Customer: Siddhant Virmani (9823042975)  
Invoice No: 653  
Product: hair mask (2 Kg)  
Subtotal: Rs. 900.00  
Discount: Rs. 45.00  
GST: Rs. 34.20  
Packing Charges: Rs. 20.00  
Membership Discount: Rs. 0.00  
Payment Mode: Card  
Total Bill: Rs. 909.20

← ↻ ⓘ File C:/Users/HP/Documents/24070521191%20-%20js%20exp/exp%20/case-study-2.html 🔍 ⭐ 🗂 Chat

Quantity (in Kg)

Rate per Kg (Rs.)

Discount (%)

GST (%)

Packing Charges (Rs.)

Payment Mode

Membership

---

Customer: Siddhant Virmani (9823042975)  
Invoice No: 653  
Product: hair mask (2 Kg)  
Subtotal: Rs. 900.00  
Discount: Rs. 45.00  
GST: Rs. 34.20  
Packing Charges: Rs. 20.00  
Membership Discount: Rs. 0.00  
Payment Mode: Card  
Total Bill: Rs. 909.20



**SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR**  
**Symbiosis International (Deemed University)**

(Established under section 3 of the UGC Act, 1956)  
Re-accredited by NAAC with 'A++' Grade | Awarded Category - I by UGC  
Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

---

## Practical 3

**Aim:-** Implement control structures and form validation; create a grading system based on user-entered marks.

**Software/Tools Required:** VS Code , Web Browser

**Experiment program code:**

**Student Marks and grade:-**

```
<!DOCTYPE html>

<html>

<head>

  <title>Student Grade</title>

  <style>

    body {

      background-color: #f4f6f9; /* Change this hex code to any color you like */

      font-family: Arial, sans-serif; /* Optional: Makes the text look cleaner */

    }

  </style>

</head>

<body>

  <h2>Student Grading System</h2>

  <h3>Student Details</h3>

  <p><b>Name:</b> Om Shendre</p>

  <p><b>PRN:</b> 24070521195</p>
```



**SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR**  
**Symbiosis International (Deemed University)**

(Established under section 3 of the UGC Act, 1956)  
Re-accredited by NAAC with 'A++' Grade | Awarded Category - I by UGC  
Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

---

**<hr>**

**<label>Enter Marks:</label>**



```
<input type="number" id="marks">
```

```
<button onclick="checkGrade()">Submit</button>
```

```
<p id="result"></p>
```

```
<script>
```

```
function checkGrade() {
```

```
    let marks = document.getElementById("marks").value;
```

```
    // Form Validation
```

```
    if (marks == "") {
```

```
        document.getElementById("result").innerHTML =
```

```
            "Please enter marks.";
```

```
        return;
```

```
    }
```

```
    marks = Number(marks);
```

```
    if (marks < 0 || marks > 100) {
```

```
        document.getElementById("result").innerHTML =
```

```
            "Marks should be between 0 and 100.";
```

```
    return;
```

```
 }
```



```
let grade = "";
```

```
// Control Structure (if-else)
```

```
if (marks >= 90) {
```

```
    grade = "A+";
```

```
}
```

```
else if (marks >= 80) {
```

```
    grade = "A";
```

```
}
```

```
else if (marks >= 70) {
```

```
    grade = "B";
```

```
}
```

```
else if (marks >= 60) {
```

```
    grade = "C";
```

```
}
```

```
else if (marks >= 50) {
```

```
    grade = "D";
```

```
}
```

```
else {
```

```
    grade = "Fail";
```

```
}
```

```
document.getElementById("result").innerHTML =
```

```
    "<b>Name:</b> Om Shendre <br>" +
```

```
    "<b>PRN:</b> 24070521195<br>" +
```

```
    "<b>Marks:</b> " + marks + "<br>" +
```

"<b>Grade:</b>" + grade;

}

</script>

</body>

</html>

**Output:-**

Student Grade

← → ↻ ⓘ File C:/Users/sit.lab3/html%201206.html

## Student Grading System

### Student Details

**Name:** Siddhant Virmani

**PRN:** 24070521206

---

Enter Marks:

**Name:** Siddhant Virmani  
**PRN:** 24070521206  
**Marks:** 80  
**Grade:** A

## Case Study 03

## Case Study Title:- Scholarship Eligibility:

A college awards scholarship based on marks

Below 50:- Not eligible

50-69:- Bronze Scholarship

70-84:- Silver Scholarship

85-94:- Gold Scholarship

95-100:- Platinum Scholarship

**Software/Tools Used:-** VScode , Web Browser

**Case Study Code:**

```
<!DOCTYPE html>
<html>

<head>
  <title>Scholarship Eligibility</title>

  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: white;
      text-align: center;
      padding: 60px;
    }

    h2 {
      color: #c21321af;
      margin: 5px 0;
    }

    h3 {
      margin-top: 30px;
      margin-bottom: 10px;
    }

    input {
width: 220px;
      padding: 10px;
      font-size: 16px;
      border: 1px solid #c0e61b;
```



```
border-radius: 4px;
outline: none;
}

button {
  display: block;
  margin: 18px auto 0;
  padding: 10px 25px;
  font-size: 16px;
  background-color: #1a5cff;
  color: white;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}

button:hover {
  background-color: #0d3fb0;
}

#result {
  margin-top: 25px;
  font-size: 18px;
  color: #444;
  line-height: 1.5;
}
</style>
</head>

<body>

<h2>om Shendre 24070521195</h2>
<h2>Scholarship Eligibility System</h2>

<h3>Enter marks:</h3>

<input type="number" id="marks">

<button onclick="checkScholarship()">Submit</button>

<p id="result"></p>

<script>

function checkScholarship() {
```

```
let marks = Number(document.getElementById("marks").value);
```

```
// Form Validation
```

```
if (document.getElementById("marks").value == "") {  
    alert("Please Enter Marks!");  
    return;  
}
```

```
if (marks < 0 || marks > 100) {  
    alert("Enter marks between 0 and 100.");  
    return;  
}
```

```
let scholarship = "";
```

```
if (marks >= 95) {  
    scholarship = "Platinum Scholarship";  
}  
else if (marks >= 85) {  
    scholarship = "Gold Scholarship";  
}  
else if (marks >= 70) {  
    scholarship = "Silver Scholarship";  
}  
else if (marks >= 50) {  
    scholarship = "Bronze Scholarship";  
}  
else {  
    scholarship = "Not Eligible";  
}
```

```
document.getElementById("result").innerHTML =  
    "Marks = " + marks +  
    "<br>Scholarship = " + scholarship;
```

```
}
```

```
</script>
```

```
</body>
```

```
</html>
```



**SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR**  
**Symbiosis International (Deemed University)**

(Established under section 3 of the UGC Act, 1956)  
Re-accredited by NAAC with 'A++' Grade | Awarded Category - I by UGC  
Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

**Output:-**

← ↻ ⓘ File C:/Users/HP/Documents/240705212061%20-%20js%20exp/exp3/Case-Study03/case%20study%203.html ☆ ☆ ⓘ ... Chat ▾

**Siddhant Virmani 24070521206**  
**Scholarship Eligibility System**

Enter marks:

98

Submit

Marks = 98  
Scholarship = Platinum Scholarship

**Aim:-** Use function types ,scope , and closure; apply try catch ;build a palindrome checker

**Software/Tools Required:** VS Code , Web Browser

**Experiment program code:**

**Palindrome checker:-**

```
<!DOCTYPE html>
<html>

<head>
  <title>Palindrome Checker</title>

  <style>
    body {
      font-family: "Times New Roman", Times, serif;
      margin: 16px;
    }

    h1 {
      font-size: 54px;
      font-weight: bold;
      margin-bottom: 40px;
    }

    input {
      width: 345px;
    }

    #result {
      margin-top: 25px;
    }
```

</style>

</head>

<body>

<h1>Palindrome Checker</h1>

<input type="text" id="userInput" value="civic">

<button onclick="checkPalindrome()">Check</button>

<p id="result"></p>

<script>

```
function checkPalindrome() {
```

```
    let input = document.getElementById("userInput").value;
```

```
    if (input == "") {
```

```
        document.getElementById("result").innerHTML =
```

```
        "Please enter a value.";
```

```
        return;
```

```
    }
```

```
    let cleanStr = input.toLowerCase().replace(/[^a-z0-9]/g, "");
```

```
    let reversedStr = cleanStr.split("").reverse().join("");
```

```
    if (cleanStr === reversedStr) {
```

```
        document.getElementById("result").innerHTML =
```

```
        "" + input + " is a palindrome! (Reversed: ' + reversedStr + ')';
```

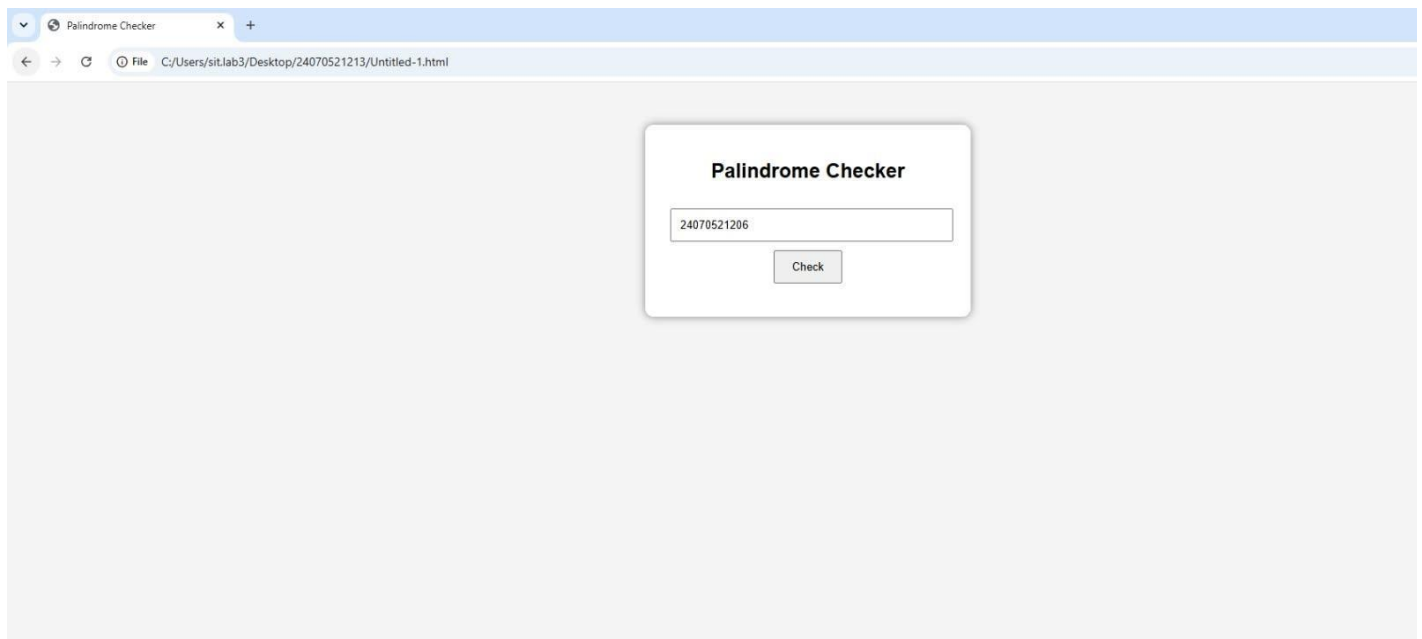
```
}  
else {  
    document.getElementById("result").innerHTML =  
        "" + input + "" is not a palindrome. (Reversed: ' + reversedStr + ');  
}  
  
}  
  
// Display the result automatically for "civic"  
checkPalindrome();
```

```
</script>
```

```
</body>
```

```
</html>
```

**Output:-**



## Case Study 04

### Real-Time Case Study: ATM Card PIN Verification Using JavaScript Functions

Case Study An ATM system verifies whether a customer's PIN is a palindrome. If the PIN reads the same forward and backward, the bank displays a special security message. To implement this, create a function to reverse the entered PIN number and another function to check whether it is a palindrome. Use function declaration, function expression, arrow function, and demonstrate scope concepts

**Software/Tools Used:-** VScode , Web Browser

**Case Study Code:-**

```
<!DOCTYPE html>
<html>
<head>
  <title>ATM PIN Verification</title>
  <style>
    /* Centers everything on the webpage */
    body {
      font-family: Arial, sans-serif;
      background-color: #f5f6fa;
      display: flex;
      justify-content: center;
      align-items: center;
      height: 100vh;
      margin: 0;
    }

    /* Clean panel layout for your practical */
    .practical-panel {
      background-color: #ffffff;
      padding: 30px;
      border-radius: 8px;
      border: 1px solid #dcdde1;
      box-shadow: 0 4px 10px rgba(0,0,0,0.05);
      text-align: center;
    }

    /* Simple styling for the text input box */
    input {
      padding: 8px;
      font-size: 16px;
      border: 1px solid #718093;
      border-radius: 4px;
      width: 150px;
    }

    /* Simple styling for the operation buttons */
    button {
      padding: 8px 15px;
      font-size: 14px;
      cursor: pointer;
      margin: 5px;
      border: 1px solid #718093;
    }
  </style>
</head>
</html>
```



```
border-radius: 4px;
background-color: #f8f9fa;
}

button:active {
  background-color: #dcdde1;
}

/* Output feedback paragraph */
#statusMessage {
  font-weight: bold;
  margin-top: 20px;
  font-size: 18px;
  color: #2f3640;
}
</style>
</head>
<body>

<div class="practical-panel">
  <h2>ATM PIN VERIFICATION</h2>

  <hr style="border: 0; border-top: 1px solid #dcdde1; margin-bottom: 20px;">

  <!-- Form layout elements -->
  <table cellpadding="8" style="margin: 0 auto;">
    <tr>
      <td><label>Enter PIN:</label></td>
      <!-- CHANGED: type="text" is now type="password" to mask typing with dots -->
      <td><input type="password" id="pinField" maxlength="6" placeholder="e.g.
1221"></td>
    </tr>
    <tr>
      <td colspan="2">
        <button onclick="verifyPin()">Verify PIN</button>
        <button onclick="clearFields()">Reset</button>
      </td>
    </tr>
  </table>

  <!-- Output Display Area -->
  <div id="statusMessage">Result: Awaiting Input...</div>
</div>

<script>
  function verifyPin() {
    var currentPin = document.getElementById("pinField").value;

    if (currentPin === "") {
```

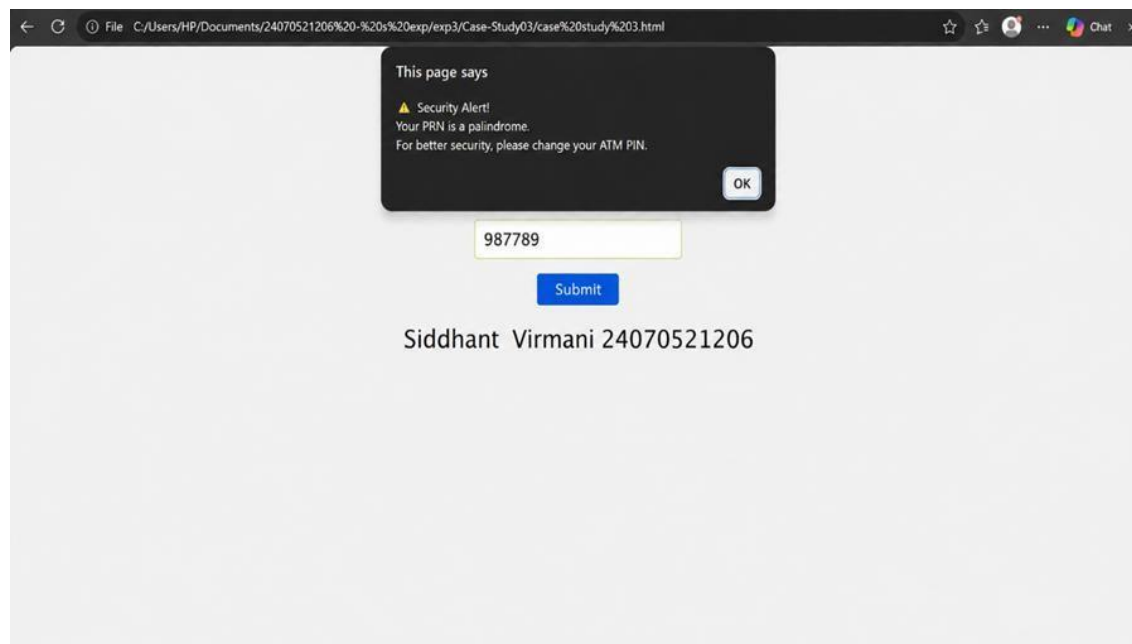
```
document.getElementById("statusMessage").innerText = "Result: Please enter a  
PIN first.";
    return;
}

// Standard reversal method
var reversedPin = currentPin.split("").reverse().join("");

if (currentPin === reversedPin) {
    document.getElementById("statusMessage").innerText = "Result: is a weak  
password! ";
} else {
    document.getElementById("statusMessage").innerText = "Result: Verification  
Successful! ";
}
}

function clearFields() {
    document.getElementById("pinField").value = "";
    document.getElementById("statusMessage").innerText = "Result: Awaiting Input...";
}
</script>
</body>
</html>
```

## Output:-





**SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR**  
**Symbiosis International (Deemed University)**

(Established under section 3 of the UGC Act, 1956)  
Re-accredited by NAAC with 'A++' Grade | Awarded Category - I by UGC  
Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

← ↻ ⓘ File C:/Users/HP/Documents/24070521206%20-%20s%20exp/case%20study%204/pin\_verification.html ☆ ⚙ 👤 ⋮ Chat ▾

**Siddhant Virmani 24070521206**  
**ATM Card PIN Verification**

**Enter your 6-digit PIN:**

982789

Submit

PIN entered = 982789  
Reversed PIN = 987289  
PIN verified. No special security message.

## Practical -05

**Title:** Apply Array Methods and Object Handling; Create a Cart Total Calculator with Discount Logic.

**Software/Tools Required:** VS Code , Web Browser

**Experiment program code:**

```
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8" />

<meta name="viewport" content="width=device-width, initial-scale=1.0" />

<title>Discount Calculator for Stationery Shop</title>

<style>

  body {

    font-family: Arial, sans-serif;

    margin: 24px;

    color: #333;

  }

  label {

    margin-right: 8px;

  }

  input {

    width: 130px;

    padding: 6px;

    margin-right: 8px;

  }

  button {

    padding: 6px 12px;
```

cursor: pointer;

}

.results {

margin-top: 18px;

max-width: 400px;

}

.summary-card {

border: 1px solid #ddd;

padding: 12px;

border-radius: 6px;

background: #fafafa;

}

.summary-card p {

margin: 6px 0;

}

.error {

color: #c00;

}

table {

width: 100%;

border-collapse: collapse;

margin-top: 10px;

}

th, td {

border: 1px solid #ccc;

padding: 6px;

text-align: left;

}

</style>

</head>

<body>

<h2>Stationery Shop Discount Calculator</h2>

<!-- Input for product name -->

<label for="product">Product:</label>

<input type="text" id="product" placeholder="Product name" />

<!-- Input for product price -->

<label for="price">Price:</label>

<input type="number" id="price" placeholder="Price" />

<button onclick="addProduct()">Add Product</button>

<!-- List of added products -->

<h3>Products:</h3>

<table id="productList">

<thead>

<tr>

<th>Product</th>

<th>Price</th>

<th>Action</th>

</tr>

</thead>

<tbody>

<!-- Products will appear here -->

</tbody>

</table>

<!-- Button to calculate discount -->

<button onclick="calculateTotal()">Calculate Total & Discount</button>

<div class="results" id="results"></div>

<script>

const products = [];

function addProduct() {

const productInput = document.getElementById('product');

const priceInput = document.getElementById('price');

const productName = productInput.value.trim();

const productPrice = parseFloat(priceInput.value);

if (!productName) {

alert('Please enter a product name.');

return;

}

if (isNaN(productPrice) || productPrice <= 0) {

alert('Please enter a valid positive price.');

return;

}

products.push({ name: productName, price: productPrice });

updateProductList();

// Clear inputs

productInput.value = '';

```
priceInput.value = "";  
}
```

```
function updateProductList() {  
  const tbody = document.getElementById('productList').getElementsByTagName('tbody')[0];  
  tbody.innerHTML = "";  
  
  products.forEach((prod, index) => {  
    const row = tbody.insertRow();  
    row.insertCell(0).innerText = prod.name;  
    row.insertCell(1).innerText = prod.price.toFixed(2);  
    const actionCell = row.insertCell(2);  
    const deleteBtn = document.createElement('button');  
    deleteBtn.innerText = 'Remove';  
    deleteBtn.onclick = () => {  
      products.splice(index, 1);  
      updateProductList();  
    };  
    actionCell.appendChild(deleteBtn);  
  });  
}
```

```
function getAutoDiscount(amount) {  
  if (amount >= 2000) return 30;  
  if (amount >= 1000) return 20;  
  return 0;  
}
```

```
function calculateTotal() {
```

```
const resultsDiv = document.getElementById('results');
```

```
if (products.length === 0) {  
    resultsDiv.innerHTML = "<p class='error'>Please add some products first.</p>";  
    return;  
}
```

```
const totalAmount = products.reduce((sum, prod) => sum + prod.price, 0);  
const discountPercentage = getAutoDiscount(totalAmount);  
const discountAmount = (totalAmount * discountPercentage) / 100;  
const finalAmount = totalAmount - discountAmount;
```

```
resultsDiv.innerHTML = `  
    <div class="summary-card">  
        <h3>Order Summary</h3>  
        <p>Total products: ${products.length}</p>  
        <p>Total amount: ${totalAmount.toFixed(2)}</p>  
        <p>Discount applied: <strong>${discountPercentage}%</strong></p>  
        <p>Discount amount: ${discountAmount.toFixed(2)}</p>  
        <p><strong>Final amount after discount: ${finalAmount.toFixed(2)}</strong></p>  
    </div>  
`;  
}
```

```
</script>
```

```
</body>
```

```
</html>
```

Discount Calculator for Stationery

File C:/Users/sit.lab3/Desktop/New%20folder/1195.html/om.html

## Stationery Shop Discount Calculator

Product:  Price:

**Products:**

| Product | Price   |
|---------|---------|
| file    | 1195.00 |
| book    | 124.00  |

### Order Summary

Total products: 2  
Total amount: 1319.00  
Discount applied: **20%**  
Discount amount: 263.80  
**Final amount after discount: 1055.20**

## RESULT/CONCLUSION:

Thus, the experiment demonstrates the use of JavaScript arrays, objects, array methods, and conditional statements to calculate a shopping cart total and apply discounts. It provides a basic understanding of data handling and calculation using JavaScript.

## Case Study-05

**Experiment Title: Study and Implementation of Arrays and Loops in JavaScript**

### 2. Software/Tools Required:

- HTML5
- CSS3
- JavaScript
- Web Browser
- VS Code / any text editor

### 3. Experiment Program Code

```
<!DOCTYPE html>

<html>

<head>

  <title>Array Operations with User Input</title>

  <style>

    body {

      font-family: Arial, sans-serif;

      background-color: #f0f8ff;

      display: flex;

      justify-content: center;

      align-items: flex-start;

      min-height: 100vh;

      margin: 0;
```

}

```
.container {  
    max-width: 800px;  
    width: 100%;  
    padding: 20px;  
    background-color: #ffffff;  
    border-radius: 12px;  
    box-shadow: 0 4px 8px rgba(0,0,0,0.1);  
    margin-top: 30px;  
}
```

}

```
h1 {  
    text-align: center;  
    color: #333;  
}
```

}

```
h2 {  
    text-align: center;  
    margin-top: 20px;  
    color: #555;  
}
```

}

```
.section {  
    margin-bottom: 20px;  
}
```

}

```
input[type=number] {  
    width: 80px;  
    padding: 6px;  
    margin-right: 8px;  
    border-radius: 6px;  
    border: 1px solid #ccc;  
}
```

}

```
button {  
    padding: 8px 12px;  
    margin-top: 8px;  
    border: none;  
    border-radius: 8px;  
    background-color: #4CAF50;  
    color: white;  
    cursor: pointer;  
    font-size: 14px;  
}
```

```
button:hover {  
    background-color: #45a049;  
}
```

```
#arrayState, #sliceResult {  
    background-color: #eef;  
    padding: 10px;  
    border-radius: 8px;  
    margin-top: 10px;  
    font-family: monospace;  
}
```

```
#sliceResult {  
    background-color: #ffe;  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="container">
```

```
    <h1>Array Operations with User Input</h1>
```

```
    <p>Initial Array: <span id="initialArrayDisplay">[10, 20, 30, 40, 50]</span></p>
```

```
<!-- Array state display -->
```

```
<h2>Array State & Results</h2>
```

```
<div id="arrayState"></div>
```

```
<!-- Push -->
```

```
<div class="section">
```

```
  <h3>Push</h3>
```

```
  <input type="number" id="pushInput" placeholder="Value to push" />
```

```
  <button onclick="pushElement()">Push</button>
```

```
</div>
```

```
<!-- Unshift -->
```

```
<div class="section">
```

```
  <h3>Unshift</h3>
```

```
  <input type="number" id="unshiftInput" placeholder="Value to unshift" />
```

```
  <button onclick="unshiftElement()">Unshift</button>
```

```
</div>
```

```
<!-- Pop -->
```

```
<div class="section">
```

```
  <h3>Pop</h3>
```

```
  <button onclick="popElement()">Pop</button>
```

```
</div>
```

```
<!-- Shift -->
```

```
<div class="section">
```

```
  <h3>Shift</h3>
```

```
  <button onclick="shiftElement()">Shift</button>
```

</div>

<!-- Slice -->

<div class="section">

    <h3>Slice</h3>

    <input type="number" id="sliceStart" placeholder="Start index" />

    <input type="number" id="sliceEnd" placeholder="End index" />

    <button onclick="sliceArray()">Slice</button>

</div>

<!-- Splice -->

<div class="section">

    <h3>Splice</h3>

    <input type="number" id="spliceStart" placeholder="Start index" />

    <input type="number" id="spliceDeleteCount" placeholder="Delete Count" />

    <input type="text" id="spliceItems" placeholder="Items to add (comma-separated)"

/>

    <button onclick="spliceArray()">Splice</button>

</div>

<!-- Slice Result -->

<h2>Slice Result</h2>

<div id="sliceResult"></div>

</div>

<script>

    // Initialize array

    let arr = [10, 20, 30, 40, 50];

```
// Function to update display
function updateDisplay() {
    document.getElementById('arrayState').innerText = 'Current Array: ' +
JSON.stringify(arr);
}

// Initial display
updateDisplay();

// Push operation
function pushElement() {
    const val = document.getElementById('pushInput').value;
    if (val !== "") {
        arr.push(Number(val));
        updateDisplay();
    }
}

// Unshift operation
function unshiftElement() {
    const val = document.getElementById('unshiftInput').value;
    if (val !== "") {
        arr.unshift(Number(val));
        updateDisplay();
    }
}

// Pop operation
function popElement() {
```

```
arr.pop();  
updateDisplay();  
}
```

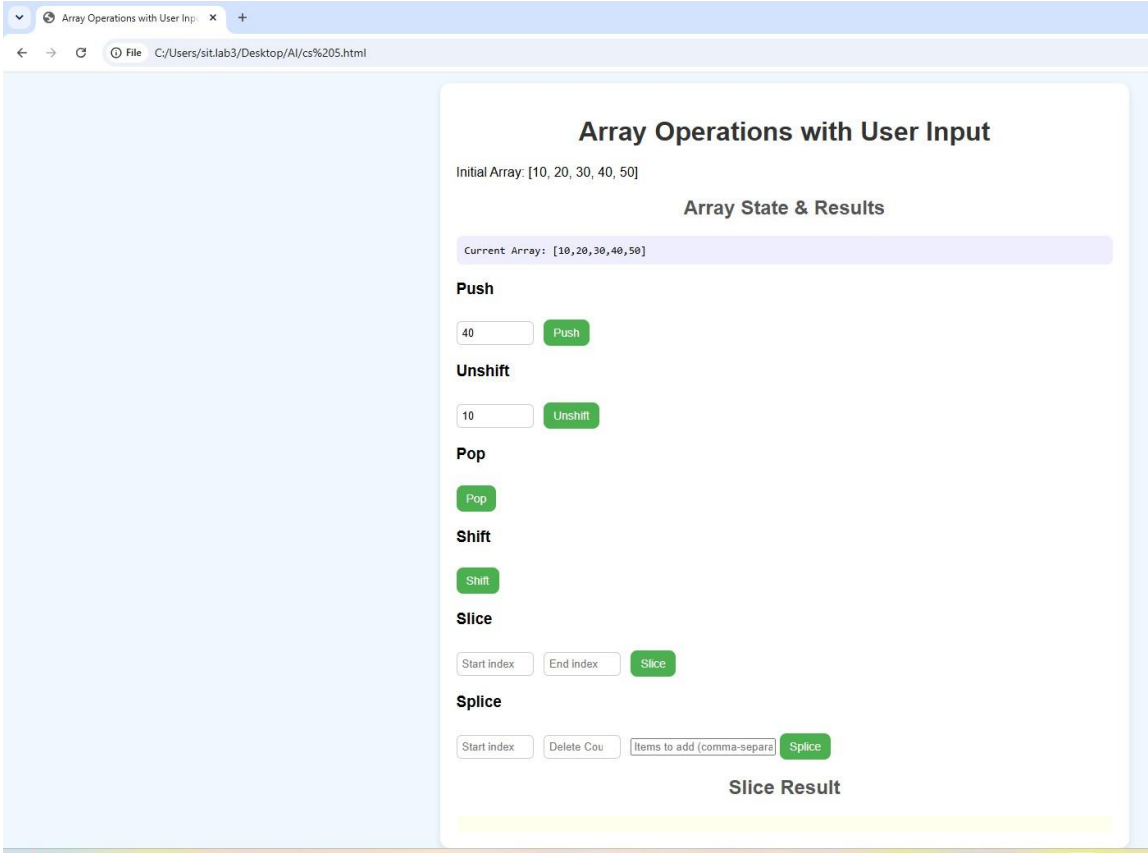
```
// Shift operation  
function shiftElement() {  
    arr.shift();  
    updateDisplay();  
}
```

```
// Slice operation  
function sliceArray() {  
    const start = parseInt(document.getElementById('sliceStart').value);  
    const end = parseInt(document.getElementById('sliceEnd').value);  
    if (!isNaN(start) && !isNaN(end)) {  
        const slicedArr = arr.slice(start, end);  
        document.getElementById('sliceResult').innerText = 'Sliced Array: ' +  
JSON.stringify(slicedArr);  
    }  
}
```

```
// Splice operation  
function spliceArray() {  
    const startIdx = parseInt(document.getElementById('spliceStart').value);  
    const deleteCount = parseInt(document.getElementById('spliceDeleteCount').value);  
    const itemsToAddStr = document.getElementById('spliceItems').value;  
    if (!isNaN(startIdx) && !isNaN(deleteCount)) {  
        const itemsToAdd = itemsToAddStr  
            .split(',')  
            .filter(item => item.trim() !== '')  
            .map(item => parseInt(item));  
        arr.splice(startIdx, deleteCount, ...itemsToAdd);  
        updateDisplay();  
    }  
}
```

```
.map(item => item.trim())  
.filter(item => item !== '')  
.map(Number);  
arr.splice(startIdx, deleteCount, ...itemsToAdd);  
updateDisplay();  
document.getElementById('sliceResult').innerText = 'Array after splice: ' +  
JSON.stringify(arr);  
}  
}  
</script>  
</body>  
</html>
```

## Output-



**Array Operations with User Input**

Initial Array: [10, 20, 30, 40, 50]

**Array State & Results**

Current Array: [10, 20, 30, 40, 50]

**Push**

**Unshift**

**Pop**

**Shift**

**Slice**

**Splice**

**Slice Result**

## Result/Conclusion:

The experiment was successfully performed to study arrays and loops in JavaScript. The maximum and minimum values of an array were successfully determined using JavaScript loops.

## Practical-06

### Experiment No.: 6

**1. Title:** Use String Functions and Regular Expressions for Email Validation, Dot Extraction, and Text Analysis.

### 2. Software/Tools Required:

1. Visual Studio Code (VS Code)
2. Google Chrome
3. HTML5
4. JavaScript

### 3. Theory:

JavaScript provides various string functions to manipulate and analyze text. Functions such as `toLowerCase()`, `toUpperCase()`, `trim()`, `split()`, `includes()`, and `replace()` are

commonly used for string processing. Regular Expressions (Regex) are patterns used to search, match, and validate text.

In this experiment, JavaScript string functions and regular expressions are used to validate an email address, extract dots from text, and perform basic text analysis. Regex helps identify whether the entered email follows a valid format, while string functions are used **to count, search, and process characters or words in the given text.**

#### **4. Output:**

Text Analysis & Email Validation

C:/Users/sit.lab3/Desktop/cs/1206.html

## Text Analysis & Email Validation

Enter your text:

NAME= Siddhant Virmani  
PRN= 24070521206

Enter email to validate:

siddhantvirmai36@gmail.com

Analyze

Validation result for "siddhantvirmai36@gmail.com": Invalid

Text Analysis Results:

-----

Extracted Emails: None

Text in Uppercase: NAME= SIDDHANT VIRMANI  
PRN= 24070521206

Text in Lowercase: name= siddhant virmani  
prn= 24070521206

Contains "Welcome": false

Text Length: 39

Number of Words: 5

Occurrences of "email": 0

## 5. Result/Conclusion:

The experiment was successfully performed. String functions and Regular Expressions were used to validate email addresses, extract dots, and analyze text. The program correctly processed the given input and displayed the required results. The objectives of the experiment were achieved successfully.

## Case Study-06

### 1. Experiment Title:

Study and Implementation of String Operations, String Manipulation Methods, and Vowel Counting in JavaScript

### 2. Software/Tools Required:

- HTML5
- CSS3
- JavaScript
- Web Browser
- VS Code / any text editor

### 3. Experiment Program Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>JavaScript String Operations</title>
<style>
* {
margin: 0;
padding: 0;
box-sizing: border-box;
}
body {
font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
min-height: 100vh;
padding: 20px;
}
.container {
```

```
max-width: 900px;
margin: 0 auto;
background: white;
border-radius: 15px;
box-shadow: 0 10px 30px rgba(0,0,0,0.3);
padding: 30px;
}
```

```
h1 {
text-align: center;
color: #667eea;
margin-bottom: 30px;
font-size: 2em;
border-bottom: 3px solid #667eea;
padding-bottom: 10px;
}
```

```
.section {
margin-bottom: 30px;
padding: 20px;
background: #f8f9fa;
border-radius: 10px;
border-left: 5px solid #667eea;
}
```

```
.section h2 {
color: #764ba2;
margin-bottom: 15px;
font-size: 1.5em;
}
```

```
label {
display: block;
```

margin-bottom: 8px;

color: #333;

font-weight: 600;

}

input[type="text"], textarea {

width: 100%;

padding: 12px;

border: 2px solid #ddd;

border-radius: 8px;

font-size: 16px;

margin-bottom: 15px;

transition: border-color 0.3s;

}

input[type="text"]:focus, textarea:focus {

outline: none;

border-color: #667eea;

}

textarea {

resize: vertical;

min-height: 100px;

}

button {

background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);

color: white;

border: none;

padding: 12px 25px;

border-radius: 8px;

cursor: pointer;

font-size: 16px;

```
font-weight: 600;
margin-right: 10px;
margin-bottom: 10px;
transition: transform 0.2s, box-shadow 0.2s;
}
button:hover {
transform: translateY(-2px);
box-shadow: 0 5px 15px rgba(102, 126, 234, 0.4);
}
button:active {
transform: translateY(0);
}
.result {
margin-top: 15px;
padding: 15px;
background: #e7f3ff;
border-radius: 8px;
border-left: 4px solid #667eea;
display: none;
}
.result.show {
display: block;
animation: fadeIn 0.5s;
}
.result strong {
color: #764ba2;
}
.string-operations {
display: grid;
```

grid-template-columns: 1fr 1fr;

gap: 15px;

margin-top: 15px;

}

.operation-box {

background: white;

padding: 15px;

border-radius: 8px;

border: 2px solid #ddd;

}

.operation-box h3 {

color: #667eea;

margin-bottom: 10px;

font-size: 1.1em;

}

@keyframes fadeIn {

from { opacity: 0; transform: translateY(-10px); }

to { opacity: 1; transform: translateY(0); }

}

.stats {

display: flex;

gap: 20px;

flex-wrap: wrap;

margin-top: 15px;

}

.stat-item {

background: white;

padding: 15px 25px;

border-radius: 8px;

```
border: 2px solid #667eea;
```

```
text-align: center;
```

```
}
```

```
.stat-item.number {
```

```
font-size: 2em;
```

```
font-weight: bold;
```

```
color: #764ba2;
```

```
}
```

```
.stat-item.label {
```

```
color: #666;
```

```
font-size: 0.9em;
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="container">
```

```
<h1> JavaScript String Operations</h1>
```

```
<!-- Case Study 1: Reverse String -->
```

```
<div class="section">
```

```
<h2>1. Reverse a String</h2>
```

```
<label for="inputString">Enter a string:</label>
```

```
<input type="text" id="inputString" placeholder="Type something to reverse...">
```

```
<button onclick="reverseString()">Reverse String</button>
```

```
<button onclick="clearReverse()">Clear</button>
```

```
<div id="reverseResult" class="result">
```

```
<strong>Reversed String:</strong> <span id="reversedText"></span>
```

```
</div>
```

```
</div>
```

```
<!-- Case Study 2: Count Vowels -->
```

```
<div class="section">
<h2>2. Count Vowels in a Paragraph</h2>
<label for="paragraph">Enter a paragraph:</label>
<textarea id="paragraph" placeholder="Type or paste your paragraph
here..."></textarea>
<button onclick="countVowels()">Count Vowels</button>
<button onclick="clearVowels()">Clear</button>
<div id="vowelResult" class="result">
<div class="stats">
<div class="stat-item">
<div class="number" id="totalVowels">0</div>
<div class="label">Total Vowels</div>
</div>
<div class="stat-item">
<div class="number" id="totalConsonants">0</div>
<div class="label">Consonants</div>
</div>
<div class="stat-item">
<div class="number" id="totalChars">0</div>
<div class="label">Total Characters</div>
</div>
</div>
<div style="margin-top: 15px;">
<strong>Vowel Details:</strong><br>
<span id="vowelDetails"></span>
</div>
</div>
</div>
<!-- String Operations Demo -->
```

<div class="section">

<h2>3. String Operations Demo (substring, indexOf, split, replace)</h2>

<label for="demoString">Enter a string for operations:</label>

<input type="text" id="demoString" placeholder="Enter text to demonstrate operations...">

<button onclick="performOperations()">Perform Operations</button>

<div id="operationsResult" class="result">

<div class="string-operations">

<div class="operation-box">

<h3>substring(0, 5)</h3>

<p id="substringResult">-</p>

</div>

<div class="operation-box">

<h3>indexOf()</h3>

<p id="indexOfResult">-</p>

</div>

<div class="operation-box">

<h3>split()</h3>

<p id="splitResult">-</p>

</div>

<div class="operation-box">

<h3>replace()</h3>

<p id="replaceResult">-</p>

</div>

</div>

</div>

</div>

</div>

<script>

### // Case Study 1: Reverse String

```
function reverseString() {  
  const input = document.getElementById('inputString').value;  
  if (input.trim() === '') {  
    alert('Please enter a string first!');  
    return;  
  }  
  const reversed = input.split('').reverse().join('');  
  document.getElementById('reversedText').textContent = reversed;  
  document.getElementById('reverseResult').classList.add('show');  
}  
  
function clearReverse() {  
  document.getElementById('inputString').value = '';  
  document.getElementById('reverseResult').classList.remove('show');  
}
```

### // Case Study 2: Count Vowels

```
function countVowels() {  
  const text = document.getElementById('paragraph').value;  
  if (text.trim() === '') {  
    alert('Please enter a paragraph first!');  
    return;  
  }  
  const vowels = text.match(/[aeiouAEIOU]/g) || [];  
  const consonants =  
  text.match(/[bcd fghjklmnpqrstvwxyzBCDFGHJ KLMNPQRSTVWXYZ]/g) || [];  
  const totalChars = text.replace(/\s/g, '').length;  
  document.getElementById('totalVowels').textContent = vowels.length;  
  document.getElementById('totalConsonants').textContent = consonants.length;  
  document.getElementById('totalChars').textContent = totalChars;
```

```
// Count individual vowels
```

```
const vowelCount = {  
'a': 0, 'e': 0, 'i': 0, 'o': 0, 'u': 0  
};  
  
text.toLowerCase().split('').forEach(char => {  
  if (vowelCount.hasOwnProperty(char)) {  
    vowelCount[char]++;  
  }  
});  
  
let details = '<ul style="margin-top: 10px; padding-left: 20px;">';  
for (let vowel in vowelCount) {  
  details += `<li><strong>${vowel.toUpperCase()}:</strong> ${vowelCount[vowel]}  
times</li>`;  
}  
details += '</ul>';  
  
document.getElementById('vowelDetails').innerHTML = details;  
document.getElementById('vowelResult').classList.add('show');  
}  
  
function clearVowels() {  
  document.getElementById('paragraph').value = '';  
  document.getElementById('vowelResult').classList.remove('show');  
}  
  
// String Operations Demo  
  
function performOperations() {  
  const str = document.getElementById('demoString').value;  
  
  if (str.trim() === '') {  
    alert('Please enter a string first!');  
    return;  
  }  
}
```

```
}  
  
// substring  
document.getElementById('substringResult').textContent = str.substring(0, 5);  
  
// indexOf (find first space)  
const spaceIndex = str.indexOf(' ');  
document.getElementById('indexOfResult').textContent = spaceIndex !== -1  
? `First space at position: ${spaceIndex}`  
: 'No space found';  
  
// split  
const words = str.split(' ');  
document.getElementById('splitResult').textContent = `Words:  
${words.length}\n[${words.join(', ')}]`;  
  
// replace  
const replaced = str.replace(/\s/g, '-');  
document.getElementById('replaceResult').textContent = `Spaces to -: ${replaced}`;  
document.getElementById('operationsResult').classList.add('show');  
}  
  
</script>  
</body>  
</html>
```

## JavaScript String Operations

### 1. Reverse a String

Enter a string:

Om

Reverse String

Clear

Reversed String: mO

### 2. Count Vowels in a Paragraph

Enter a paragraph:

Hell this is om

Copy

Count Vowels

Clear

5

Total Vowels

7

Consonants

15

Total Characters

### Result/Conclusion:

The experiment was successfully performed to study string operations and manipulation methods in JavaScript. The string was successfully reversed and the number of vowels was calculated.

## Experiment No.: 7

**Title:** Perform DOM Traversal and Updates; Develop a To-Do List App with Add, Edit, and Delete Features.

### 2. Software/Tools Required:

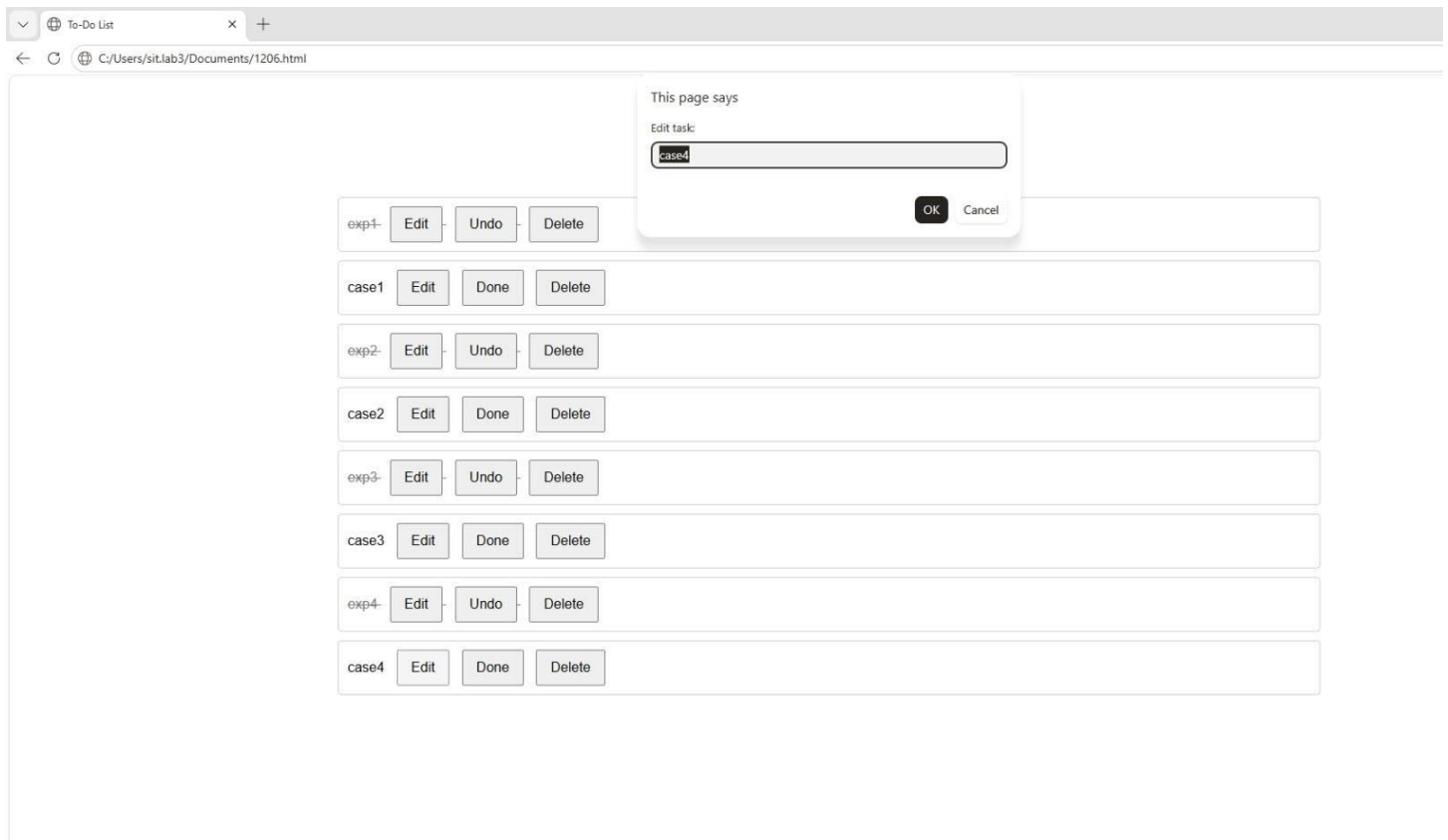
5. Visual Studio Code (VS Code)
6. Google Chrome
7. HTML5
8. JavaScript

### 3. Theory:

The Document Object Model (DOM) represents an HTML document as a tree of elements. JavaScript can access and modify HTML elements dynamically using DOM methods such as `getElementById()`, `querySelector()`, `createElement()`, and `remove()`.

In this experiment, a To-Do List application is developed using HTML, CSS, and JavaScript. Users can enter a task and add it to the list. Each task contains a checkbox along with Edit and Delete buttons. The DOM is updated dynamically whenever a task is added, edited, deleted, or marked as completed. This demonstrates DOM traversal, element creation, modification, and removal.

### 4. Output:



## 5. Result/Conclusion:

The experiment was successfully performed. The To-Do List application demonstrated DOM traversal and dynamic DOM updates using JavaScript. Tasks were successfully displayed with checkboxes and Edit and Delete options. The application allowed tasks to be added, modified, deleted, and marked as completed. Thus, the objectives of the experiment were achieved successfully.

## Case Study-07

## 1. Experiment Title:

Study and Implementation of Event Handling, Form Validation, and Dynamic DOM Manipulation in JavaScript

## 2. Software/Tools Required:

- HTML5
- CSS3
- JavaScript
- Web Browser
- VS Code / any text editor

## 2. Experiment Program Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>JavaScript Event Handling & Form Validation</title>
<style>
* {
margin: 0;
padding: 0;
box-sizing: border-box;
}
body {
font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
background: linear-gradient(135deg, #f093fb 0%, #f5576c 100%);
min-height: 100vh;
padding: 20px;
}
.container {
```

```
max-width: 800px;
margin: 0 auto;
background: white;
border-radius: 15px;
box-shadow: 0 10px 30px rgba(0,0,0,0.3);
padding: 40px;
}

h1 {
text-align: center;
color: #f5576c;
margin-bottom: 10px;
font-size: 2.2em;
}

.subtitle {
text-align: center;
color: #666;
margin-bottom: 30px;
font-size: 1.1em;
}

.form-group {
margin-bottom: 25px;
}

label {
display: block;
margin-bottom: 8px;
color: #333;
font-weight: 600;
font-size: 14px;
}
```

```
label.required {
color: #f5576c;
}
input[type="text"],
input[type="email"],
input[type="password"],
input[type="tel"],
input[type="number"],
select,
textarea {
width: 100%;
padding: 12px 15px;
border: 2px solid #ddd;
border-radius: 8px;
font-size: 15px;
transition: all 0.3s;
}
input[type="text"]:focus,
input[type="email"]:focus,
input[type="password"]:focus,
input[type="tel"]:focus,
input[type="number"]:focus,
select:focus,
textarea:focus {
outline: none;
border-color: #f5576c;
box-shadow: 0 0 0 3px rgba(245, 87, 108, 0.1);
}
input.error {
```

border-color: #f5576c;

background-color: #fff5f5;

}

input.success {

border-color: #10b981;

background-color: #f0fdf4;

}

.error-message {

color: #f5576c;

font-size: 13px;

margin-top: 5px;

display: none;

}

.error-message.show {

display: block;

}

.hint {

color: #666;

font-size: 12px;

margin-top: 5px;

font-style: italic;

}

.button-group {

display: flex;

gap: 15px;

margin-top: 30px;

}

button {

flex: 1;

padding: 14px 28px;

border: none;

border-radius: 8px;

font-size: 16px;

font-weight: 600;

cursor: pointer;

transition: all 0.3s;

}

.submit-btn {

background: linear-gradient(135deg, #5456e7 0%, #f5576c 100%);

color: white;

}

.submit-btn:hover {

transform: translateY(-2px);

box-shadow: 0 5px 15px rgba(245, 87, 108, 0.4);

}

.reset-btn {

background: #6b7280;

color: white;

}

.reset-btn:hover {

background: #4b5563;

}

.event-log {

margin-top: 30px;

padding: 20px;

background: #f8f9fa;

border-radius: 10px;

border-left: 5px solid #f5576c;

}

```
.event-log h3 {  
color: #f5576c;  
margin-bottom: 15px;
```

}

```
.log-entry {  
padding: 8px 12px;  
background: white;  
margin-bottom: 8px;  
border-radius: 5px;  
font-size: 14px;  
border-left: 3px solid #f093fb;  
animation: slideIn 0.3s;
```

}

```
@keyframes slideIn {  
from {  
opacity: 0;  
transform: translateX(-20px);  
}
```

```
to {  
opacity: 1;  
transform: translateX(0);  
}  
}
```

```
.success-message {  
background: #10b981;  
color: white;  
padding: 20px;  
border-radius: 10px;
```

```
text-align: center;
margin-top: 20px;
display: none;
}

.success-message.show {
display: block;
animation: fadeIn 0.5s;
}

@keyframes fadeIn {
from { opacity: 0; }
to { opacity: 1; }
}

.char-count {
text-align: right;
font-size: 12px;
color: #666;
margin-top: 5px;
}

.char-count.warning {
color: #f59e0b;
}

.char-count.danger {
color: #f5576c;
}

fieldset {
border: 2px solid #ddd;
border-radius: 8px;
padding: 20px;
margin-bottom: 20px;
```

```
}  
  
legend {  
  color: #f5576c;  
  font-weight: 600;  
  padding: 0 10px;  
}  
  
.inline-fields {  
  display: grid;  
  grid-template-columns: 1fr 1fr;  
  gap: 15px;  
}  
  
@media (max-width: 600px) {  
  .inline-fields {  
    grid-template-columns: 1fr;  
  }  
  .button-group {  
    flex-direction: column;  
  }  
}  
  
</style>  
</head>  
<body>  
  <div class="container">  
    <h1> Registration Form</h1>  
    <p class="subtitle">Event Handling & Form Validation Demo</p>  
    <form id="registrationForm" novalidate>  
      <fieldset>  
        <legend>Personal Information</legend>  
        <div class="inline-fields">
```

```
<div class="form-group">
```

```
<label for="firstName">First Name <span class="required">*</span></label>
```

```
<input type="text" id="firstName" name="firstName" placeholder="Enter first name">
```

```
<div class="error-message" id="firstNameError">Please enter your first name</div>
```

```
</div>
```

```
<div class="form-group">
```

```
<label for="lastName">Last Name <span class="required">*</span></label>
```

```
<input type="text" id="lastName" name="lastName" placeholder="Enter last name">
```

```
<div class="error-message" id="lastNameError">Please enter your last name</div>
```

```
</div>
```

```
</div>
```

```
<div class="form-group">
```

```
<label for="email">Email Address <span class="required">*</span></label>
```

```
<input type="email" id="email" name="email" placeholder="example@email.com">
```

```
<div class="error-message" id="emailError">Please enter a valid email address</div>
```

```
<div class="hint">We'll never share your email with anyone</div>
```

```
</div>
```

```
<div class="form-group">
```

```
<label for="phone">Phone Number <span class="required">*</span></label>
```

```
<input type="tel" id="phone" name="phone" placeholder="+1 (555) 123-4567">
```

```
<div class="error-message" id="phoneError">Please enter a valid 10-digit phone  
number</div>
```

```
</div>
```

```
</fieldset>
```

```
<fieldset>
```

```
<legend>Account Details</legend>
```

```
<div class="form-group">
```

```
<label for="password">Password <span class="required">*</span></label>
```

```
<input type="password" id="password" name="password" placeholder="Create a password">
```

```
<div class="error-message" id="passwordError">Password must be at least 8 characters with uppercase, lowercase, and number</div>
```

```
<div class="hint">Minimum 8 characters, include uppercase, lowercase, and number</div>
```

```
</div>
```

```
<div class="form-group">
```

```
<label for="confirmPassword">Confirm Password <span class="required">*</span></label>
```

```
<input type="password" id="confirmPassword" name="confirmPassword" placeholder="Re-enter password">
```

```
<div class="error-message" id="confirmPasswordError">Passwords do not match</div></div>
```

```
<div class="form-group">
```

```
<label for="country">Country <span class="required">*</span></label>
```

```
<select id="country" name="country">
```

```
<option value="">Select your country</option>
```

```
<option value="usa">United States</option>
```

```
<option value="uk">United Kingdom</option>
```

```
<option value="canada">Canada</option>
```

```
<option value="australia">Australia</option>
```

```
<option value="india">India</option>
```

```
<option value="other">Other</option>
```

```
</select>
```

```
<div class="error-message" id="countryError">Please select a country</div>
```

```
</div>
```

```
</fieldset>
```

```
<fieldset>
```

<legend>Additional Information</legend>

<div class="form-group">

<label for="bio">Brief Bio</label>

<textarea id="bio" name="bio" rows="4" placeholder="Tell us about yourself (max 200 characters)"></textarea>

<div class="char-count" id="charCount">0/200 characters</div>

</div>

<div class="form-group">

<label for="age">Age</label>

<input type="number" id="age" name="age" placeholder="Your age" min="1" max="120">

<div class="error-message" id="ageError">Please enter a valid age between 1 and 120</div>

</div>

</fieldset>

<div class="button-group">

<button type="submit" class="submit-btn">Submit Registration</button>

<button type="reset" class="reset-btn">Reset Form</button>

</div>

</form>

<div class="success-message" id="successMessage">

<h2>  Registration Successful!</h2>

<p>Thank you for registering. We've received your information.</p>

</div>

<div class="event-log">

<h3>  Event Log</h3>

<div id="logContainer">

<div class="log-entry">Form loaded. Start filling the form...</div>

</div>

</div>

</div>

<script>

// Get form elements

const form = document.getElementById('registrationForm');

const logContainer = document.getElementById('logContainer');

const successMessage = document.getElementById('successMessage');

// Event logging function

function logEvent(eventName, fieldName = "") {

const timestamp = new Date().toLocaleTimeString();

const entry = document.createElement('div');

entry.className = 'log-entry';

entry.textContent = `[\${timestamp}] \${eventName}\${fieldName ? ' - ' + fieldName : ''}`;

logContainer.insertBefore(entry, logContainer.firstChild);

// Keep only last 10 entries

while (logContainer.children.length > 10) {

logContainer.removeChild(logContainer.lastChild);

}

}

// Validation functions

function validateEmail(email) {

const re = /^[^\s@]+@[^\s@]+\.[^\s@]+\$/;

return re.test(email);

}

function validatePhone(phone) {

const re = /^[d\s\-\+\(\)]{10,}\$/;

return re.test(phone.replace(/\s/g, ""));

}

function validatePassword(password) {

```
const re = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d){8,}$/;

return re.test(password);

}

function showError(fieldId, show = true) {
  const field = document.getElementById(fieldId);
  const errorDiv = document.getElementById(fieldId + 'Error');
  if (show) {
    field.classList.add('error');
    field.classList.remove('success');
    errorDiv.classList.add('show');
  } else {
    field.classList.remove('error');
    field.classList.add('success');
    errorDiv.classList.remove('show');
  }
}

// Focus event
const inputs = form.querySelectorAll('input, select, textarea');
inputs.forEach(input => {
  input.addEventListener('focus', function() {
    logEvent('Focus', this.name || this.id);
    this.classList.remove('error');
  });
});

// Blur event
input.addEventListener('blur', function() {
  logEvent('Blur', this.name || this.id);
  validateField(this.id);
});

// Change event
```

```
input.addEventListener('change', function() {
    logEvent('Change', this.name || this.id);
});
// Input event (for real-time validation)
input.addEventListener('input', function() {
    if (this.classList.contains('error')) {
        validateField(this.id);
    }
});
});
// Character count for bio
const bioField = document.getElementById('bio');
const charCount = document.getElementById('charCount');
bioField.addEventListener('input', function() {
    const length = this.value.length;
    charCount.textContent = `${length}/200 characters`;
    charCount.classList.remove('warning', 'danger');
    if (length > 180) {
        charCount.classList.add('danger');
    } else if (length > 150) {
        charCount.classList.add('warning');
    }
    if (length > 200) {
        this.value = this.value.substring(0, 200);
        charCount.textContent = '200/200 characters (max reached)';
    }
});
// Individual field validation
function validateField(fieldId) {
```

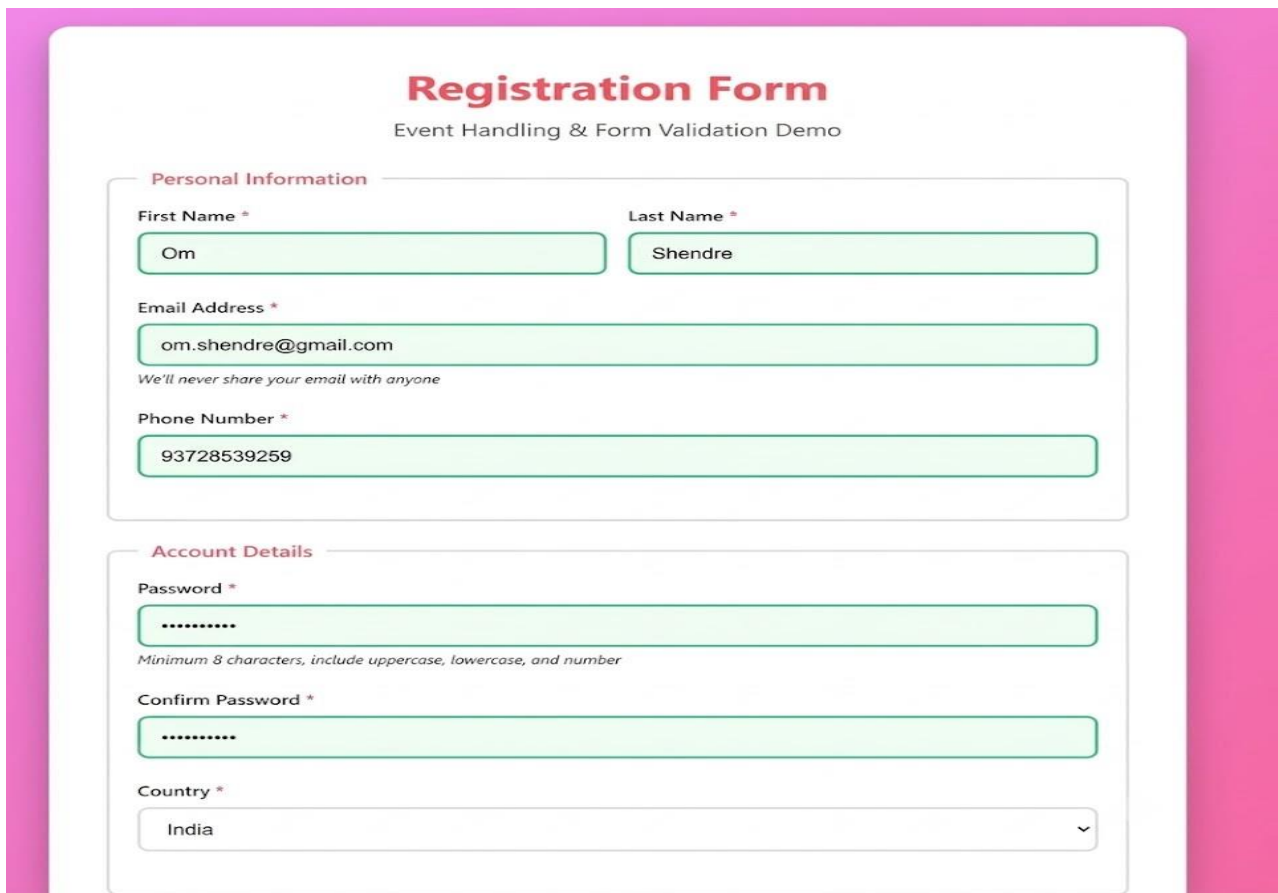
```
const field = document.getElementById(fieldId);
const value = field.value.trim();
let isValid = true;
switch(fieldId) {
case 'firstName':
case 'lastName':
    isValid = value.length > 0;
    break;
case 'email':
    isValid = validateEmail(value);
    break;
case 'phone':
    isValid = validatePhone(value);
    break;
case 'password':
    isValid = validatePassword(value);
    break;
case 'confirmPassword':
    const password = document.getElementById('password').value;
    isValid = value === password && value.length > 0;
    break;
case 'country':
    isValid = value !== '';
    break;
case 'age':
    const age = parseInt(value);
    isValid = value === '' || (age >= 1 && age <= 120);
    break;
}
```

```
showError(fieldId, !isValid);
return isValid;
}
// Form submission
form.addEventListener('submit', function(e) {
  e.preventDefault();
  logEvent('Submit');
  // Validate all fields
  const fields = ['firstName', 'lastName', 'email', 'phone', 'password', 'confirmPassword',
    'country'];
  let isValid = true;
  fields.forEach(fieldId => {
    if (!validateField(fieldId)) {
      isValid = false;
    }
  });
  // Validate age if filled
  const ageField = document.getElementById('age');
  if (ageField.value !== "" && !validateField('age')) {
    isValid = false;
  }
  if (isValid) {
    // Show success message
    successMessage.classList.add('show');
    logEvent('Success', 'Form submitted successfully');
    // Scroll to success message
    successMessage.scrollIntoView({ behavior: 'smooth', block: 'center' });
    // Hide success message after 5 seconds
    setTimeout(() => {
```

```
successMessage.classList.remove('show');
}, 5000);
// In a real application, you would submit the form data here
console.log('Form data:', new FormData(form));
} else {
  logEvent('Error', 'Form validation failed');
  alert('Please fix the errors in the form before submitting.');
```

```
}
});
// Form reset
form.addEventListener('reset', function() {
  logEvent('Reset');
  successMessage.classList.remove('show');
  // Clear all error states
  inputs.forEach(input => {
    input.classList.remove('error', 'success');
  });
  // Hide all error messages
  const errorMessages = document.querySelectorAll('.error-message');
  errorMessages.forEach(msg => msg.classList.remove('show'));
  // Reset char count
  charCount.textContent = '0/200 characters';
  charCount.classList.remove('warning', 'danger');
  setTimeout(() => {
    alert('Form has been reset');
  }, 100);
});
// Add enter key logging
document.addEventListener('keypress', function(e) {
```

```
if (e.key === 'Enter' && e.target.tagName === 'INPUT') {  
    logEvent('Enter Key', e.target.name || e.target.id);  
}  
});  
</script>  
</body>  
</html>
```



The image shows a web form titled "Registration Form" with the subtitle "Event Handling & Form Validation Demo". The form is divided into two sections: "Personal Information" and "Account Details".

**Personal Information**

- First Name \***: Input field with value "Om".
- Last Name \***: Input field with value "Shendre".
- Email Address \***: Input field with value "om.shendre@gmail.com". Below the field is a note: "We'll never share your email with anyone".
- Phone Number \***: Input field with value "93728539259".

**Account Details**

- Password \***: Input field with masked characters "\*\*\*\*\*". Below the field is a note: "Minimum 8 characters, include uppercase, lowercase, and number".
- Confirm Password \***: Input field with masked characters "\*\*\*\*\*".
- Country \***: Dropdown menu with "India" selected.

### Result/Conclusion:

The experiment was successfully performed to study JavaScript event handling, form validation, and dynamic DOM manipulation. The form was validated and the output message was dynamically displayed using JavaScript.

**1. Title:** Access and Validate Form Fields Using Events; Design a Gym Admission Form with Live Input Check.

**2. Software/Tools Required:**

Visual Studio Code (VS Code)

Google Chrome

HTML5

JavaScript

**3. Theory:**

JavaScript events are actions that occur on a webpage, such as input, change, focus, blur, and submit. Event handlers are used to execute JavaScript code when these events occur. DOM methods such as `getElementById()` are used to access and modify form fields. In this experiment, a Gym Admission Form is designed using HTML and CSS. JavaScript is used to access the form fields and perform live validation while the user enters information. The input event checks the entered values immediately and displays appropriate validation messages. Form submission is also validated to ensure that the required details are entered correctly before the form is submitted.

**4. Output:**

Gym Membership Registration

file:///C:/Users/SIT.LAB3/Desktop/java.6/1206.html

**Gym Membership Registration**

**Full Name:**

**Email:**

**Phone Number:**

**Age:**

**Membership Type:**

**Start Date:**

Thank you, Siddhant Virmani! Your Yearly membership for age 19 starting on 2026-09-12 has been registered.

Gym Membership Registration

file:///C:/Users/SIT.LAB3/Desktop/java.6/1206.html

**Gym Membership Registration**

**Full Name:**

**Email:**

**Phone Number:**

**Age:**

**Membership Type:**

**Start Date:**

Thank you, Siddhant Virmani! Your Yearly membership for age 19 starting on 2026-09-12 has been registered.

Please include an '@' in the email address. 'siddhant36gmail.com' is missing an '@'.

## 5. Result/Conclusion:

The experiment was successfully performed. The Gym Admission Form demonstrated the use of JavaScript events to access and validate form fields. Live input checks were performed while entering data, and validation messages were displayed accordingly. The form successfully accepted valid information and prevented invalid or incomplete data from being submitted. Thus, the objectives of the experiment were achieved successfully.

## 1. Experiment Title:

Design and Implementation of a Multi-Section Job Application Portal Using HTML5, CSS Grid, and JavaScript

## 2. Software/Tools Required:

- HTML5
- CSS3
- JavaScript
- Web Browser
- VS Code / any text editor

## 3. Experiment Program Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8" />
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
<title>Job Application Form</title>
<style>
body {
margin: 0;
font-family: Arial, sans-serif;
background: #f0f0f0; /* Light background for contrast */
display: flex;
justify-content: center;
align-items: center;
min-height: 100vh;
}
#jobApplicationForm {
background: #fff;
padding: 20px 30px;
```

border-radius: 10px;

box-shadow: 0 4px 15px rgba(0,0,0,0.2);

max-width: 600px;

width: 100%;

}

h2 {

text-align: center;

margin-bottom: 20px;

color: #333;

}

h3 {

margin-top: 20px;

margin-bottom: 10px;

color: #555;

}

label {

display: block;

margin-top: 10px;

font-weight: 600;

}

input, select {

width: 100%;

padding: 8px 12px;

margin-top: 4px;

border-radius: 4px;

border: 2px solid #ccc;

font-size: 1em;

box-sizing: border-box;



transition: border-color 0.2s;

}

input:focus, select:focus {

border-color: #007BFF;

outline: none;

}

span {

color: red;

font-size: 0.85em;

height: 1em;

display: block;

margin-top: 2px;

}

#submitBtn {

background-color: #007BFF;

color: white;

padding: 10px 20px;

border: none;

border-radius: 5px;

font-size: 1em;

cursor: pointer;

margin-top: 20px;

}

#submitBtn:disabled {

background-color: #aaa;

cursor: not-allowed;

}

/\* Optional: hover effects for the button \*/

```
#submitBtn:hover {  
    background-color: #0056b3;  
}
```

</style>

</head>

<body>

<form id="jobApplicationForm">

<!-- Personal Information -->

<h2>Job Application Form</h2>

<h3>Personal Information</h3>

<label>Name:

<input type="text" id="name" required>

<span id="nameError"></span>

</label>

<label>Email:

<input type="email" id="email" required>

<span id="emailError"></span>

</label>

<label>Phone:

<input type="tel" id="phone" required>

<span id="phoneError"></span>

</label>

<!-- Educational Qualification -->

### Educational Qualification</h3>

<label>Highest Degree:

<select id="degree" required>  
    <option value="">Select</option>  
    <option value="Bachelors">Bachelors</option>  
    <option value="Masters">Masters</option>  
    <option value="PhD">PhD</option>  
</select>  
<span id="degreeError"></span>

</label>

<label>Institution Name:

<input type="text" id="institution" required>  
<span id="institutionError"></span>

</label>

<label>Year of Graduation:

<input type="number" id="graduationYear" required>  
<span id="graduationYearError"></span>

</label>

<!-- Technical Skills -->

### Technical Skills</h3>

<label>Skills:

<input type="text" id="skills" required>  
<span id="skillsError"></span>

</label>

<label>Proficiency:

<select id="proficiency" required>

<option value="">Select</option>

<option value="Beginner">Beginner</option>

<option value="Intermediate">Intermediate</option>

<option value="Expert">Expert</option>

</select>

<span id="proficiencyError"></span>

</label>

<!-- Professional Information -->

<h3>Professional Information</h3>

<label>Current Employer:

<input type="text" id="employer">

<span id="employerError"></span>

</label>

<label>Years of Experience:

<input type="number" id="experience" required>

<span id="experienceError"></span>

</label>

<button type="submit" id="submitBtn" disabled>Submit Application</button>

</form>

<script>

const form = document.getElementById('jobApplicationForm');

```
const submitBtn = document.getElementById('submitBtn');
```

```
const validators = {  
  name: () => /^[A-Za-z\s]+$/.test(document.getElementById('name').value),  
  email: () =>  
    /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(document.getElementById('email').value),  
  phone: () => /^\d{10}$/.test(document.getElementById('phone').value),  
  institution: () => document.getElementById('institution').value.trim() !== '',  
  graduationYear: () => {  
    const year = parseInt(document.getElementById('graduationYear').value);  
    const currentYear = new Date().getFullYear();  
    return year >= 1900 && year <= currentYear;  
  },  
  skills: () => document.getElementById('skills').value.trim() !== '',  
  experience: () => {  
    const exp = parseInt(document.getElementById('experience').value);  
    return !isNaN(exp) && exp >= 0;  
  },  
  degree: () => document.getElementById('degree').value !== '',  
  proficiency: () => document.getElementById('proficiency').value !== ''  
};
```

```
function validateField(id, validator, errorId) {  
  const value = document.getElementById(id).value.trim();  
  const isValid = validator();  
  const errorSpan = document.getElementById(errorId);  
  if (!isValid) {
```

```
errorSpan.textContent = 'Invalid or required.';
```

```
} else {
```

```
    errorSpan.textContent = "";
```

```
}
```

```
return isValid;
```

```
}
```

```
function validateForm() {
```

```
    const validName = validateField('name', validators.name, 'nameError');
```

```
    const validEmail = validateField('email', validators.email, 'emailError');
```

```
    const validPhone = validateField('phone', validators.phone, 'phoneError');
```

```
    const validInstitution = validateField('institution', validators.institution,  
'institutionError');
```

```
    const validYear = validateField('graduationYear', validators.graduationYear,  
'graduationYearError');
```

```
    const validSkills = validateField('skills', validators.skills, 'skillsError');
```

```
    const validExperience = validateField('experience', validators.experience,  
'experienceError');
```

```
    const validDegree = validateField('degree', validators.degree, 'degreeError');
```

```
    const validProficiency = validateField('proficiency', validators.proficiency,  
'proficiencyError');
```

```
const allValid = validName && validEmail && validPhone && validInstitution &&  
    validYear && validSkills && validExperience && validDegree &&  
validProficiency;
```

```
submitBtn.disabled = !allValid;
```

}

```
// Add event listeners for real-time validation
document.querySelectorAll('#jobApplicationForm input, #jobApplicationForm
select').forEach(elem => {
    elem.addEventListener('input', validateForm);
});

form.addEventListener('submit', function(e) {
    e.preventDefault();
    validateForm();
    if (!submitBtn.disabled) {
        alert('Application submitted successfully!');
        form.reset();
        submitBtn.disabled = true;
        // Clear error messages
        document.querySelectorAll('span').forEach(span => span.textContent = '');
    }
});
</script>

</body>
</html>
```

Output-

Job Application Form

**Personal Information**

**Name:**

**Email:**  
  
**Invalid or required.**

**Phone:**  
  
**Invalid or required.**

**Educational Qualification**

**Highest Degree:**

**Institution Name:**

**Year of Graduation:**

**Technical Skills**

**Skills:**

## 5. Result/Conclusion:

The experiment was successfully performed to design a multi-section job application portal using semantic HTML5 form elements, CSS Grid layout, and JavaScript form submission handling. The form was divided into logical sections and successfully validated and submitted using JavaScript.

## Practical-09

**Title:** Implement localStorage and sessionStorage to save user preferences like theme settings.

## 2. Software/Tools Required:

Visual Studio Code (VS Code)

Google Chrome

HTML5

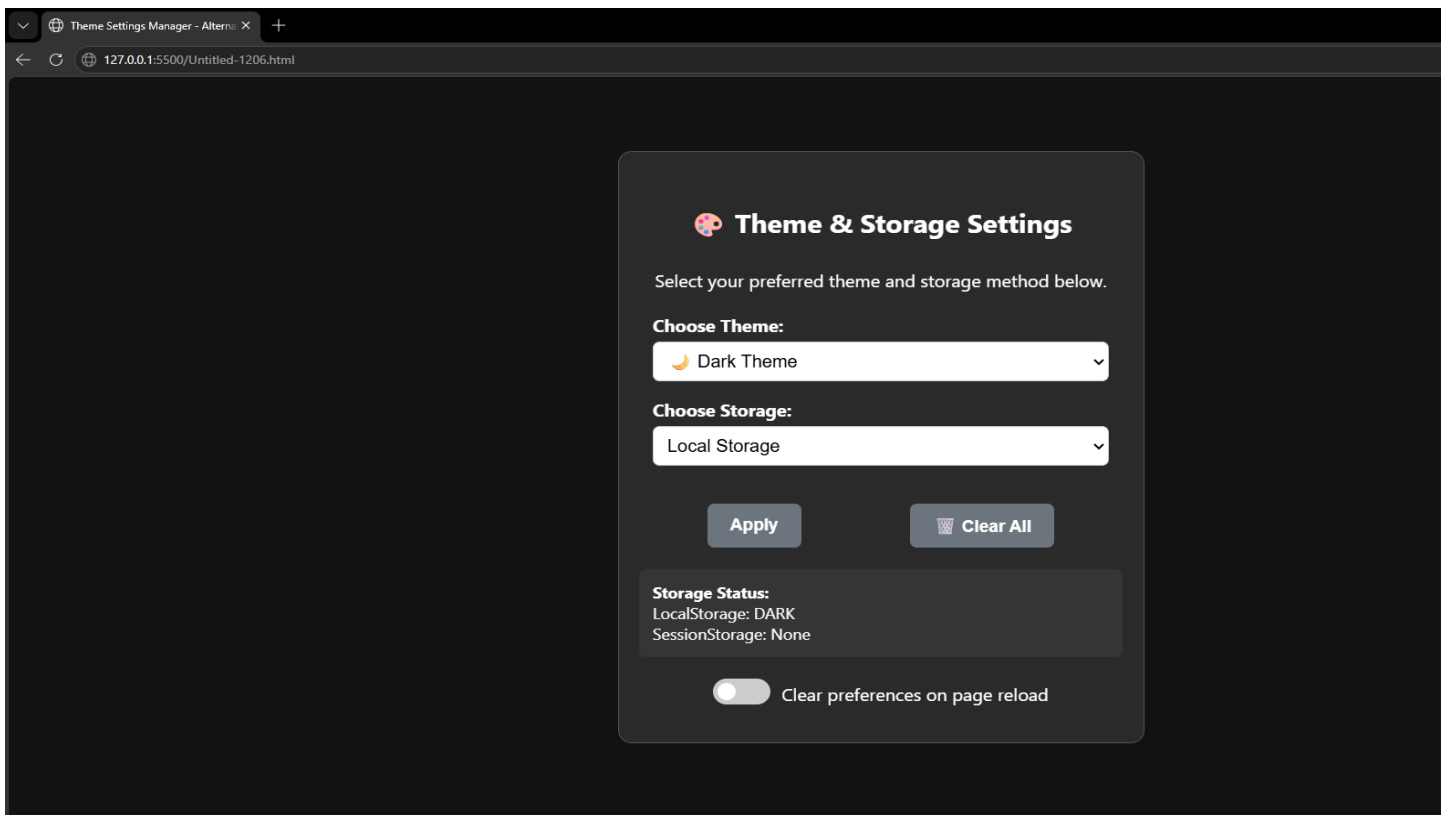
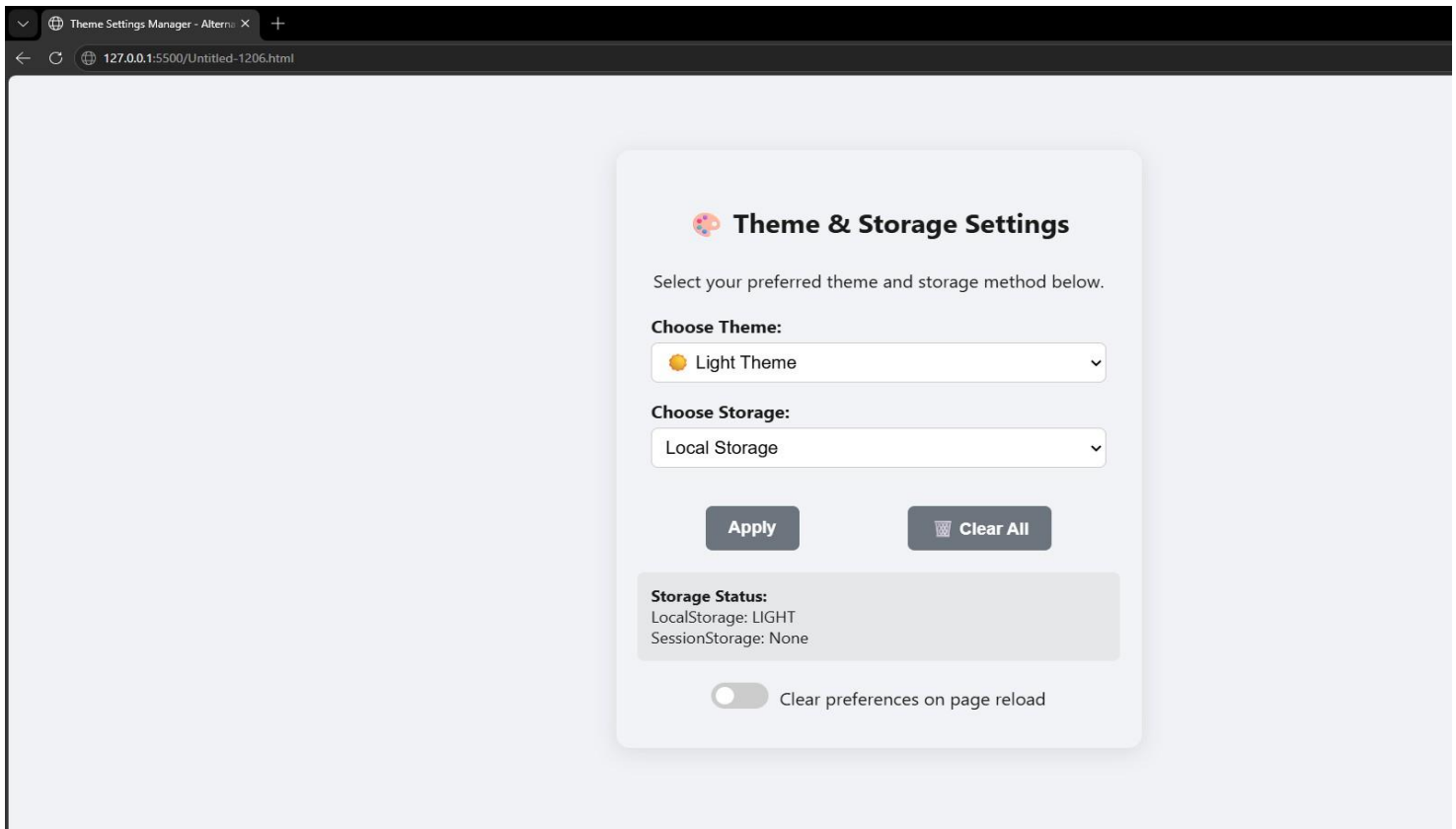
JavaScript

## 3. Theory:

Web Storage allows websites to store data in the user's browser. JavaScript provides two types of web storage: **Local Storage** and **Session Storage**. Local Storage stores data without an expiration time, while Session Storage stores data only for the current browser session.

In this experiment, a Theme Settings webpage is created to allow the user to select a preferred theme such as **Light** or **Dark**. JavaScript is used to save the selected theme using `localStorage` and `sessionStorage`. The `setItem()` method is used to store preferences, while `getItem()` can be used to retrieve stored preferences. The `clear()` method is used to remove stored preferences. A message is displayed after successfully saving the selected theme.

## 4. Output:



Theme Settings Manager - Altern... X +

127.0.0.1:5500/Untitled-1206.html

 **Theme & Storage Settings**

Select your preferred theme and storage method below.

**Choose Theme:**

☀️ Light Theme ▼

**Choose Storage:**

Local Storage ▼

Apply

🗑️ Clear All

**Storage Status:**

LocalStorage: LIGHT  
SessionStorage: None

☐

 Clear preferences on page reload

Theme Settings Manager - Altern... X +

127.0.0.1:5500/Untitled-1206.html

 **Theme & Storage Settings**

Select your preferred theme and storage method below.

**Choose Theme:**

🌙 Dark Theme ▼

**Choose Storage:**

Local Storage ▼

Apply

🗑️ Clear All

**Storage Status:**

LocalStorage: DARK  
SessionStorage: None

☐

 Clear preferences on page reload

## **. Result/Conclusion:**

The experiment was successfully performed. The To-Do List application demonstrated DOM traversal and dynamic DOM updates using JavaScript. Tasks were successfully displayed with checkboxes and Edit and Delete options. The application allowed tasks to be added, modified, deleted, and marked as completed. Thus, the objectives of the experiment were achieved successfully.

## **Practical-10**

**Title:** Load and Display JSON Data Using Fetch() and jQuery \$.getJSON() and Render Results Dynamically in a Table.

### **2. Software/Tools Required:**

Visual Studio Code (VS Code)

Google Chrome

HTML5

JavaScript

### **2. Theory:**

JSON (JavaScript Object Notation) is a lightweight format used to store and exchange data. JavaScript provides the **Fetch API** to request and process data asynchronously. jQuery provides the \$.getJSON() method to load JSON data easily from a file or URL. In this experiment, JSON data is loaded using both the Fetch API and jQuery \$.getJSON(). The retrieved JSON data is converted into JavaScript objects and processed dynamically. JavaScript is then used to create table rows and display the JSON data in an HTML table. This demonstrates asynchronous data loading, JSON handling, jQuery, and dynamic DOM manipulation.

4. Output:

Student JSON Data Table

File C:/Users/sit.lab3/Desktop/exp%2010/1195.html

### Student JSON Data

Load Using fetch()

Load Using \$.getJSON()

Data loaded successfully using fetch()

| PRN  | Name             | Email              |
|------|------------------|--------------------|
| 1195 | Om Shendre       | om@gmail.com       |
| 1193 | kanak Sawarkar   | kanak@gmail.com    |
| 1208 | Arnav wandile    | arnav@gmail.com    |
| 1181 | preetish notwani | preetish@gmail.com |
| 1206 | siddhant virmani | siddhant@gmail.com |

Student JSON Data Table

File C:/Users/sit.lab3/Desktop/exp%2010/1195.html

## Student JSON Data

Load Using fetch()

Load Using \$.getJSON()

Data loaded successfully using \$.getJSON()

| PRN  | Name             | Email              |
|------|------------------|--------------------|
| 1195 | Om Shendre       | om@gmail.com       |
| 1193 | kanak Sawarkar   | kanak@gmail.com    |
| 1208 | Arnav wandile    | arnav@gmail.com    |
| 1181 | preetish notwani | preetish@gmail.com |
| 1206 | siddhant virmani | siddhant@gmail.com |

## 5. Result/Conclusion:

The experiment was successfully performed. JSON data was loaded using the Fetch API and jQuery \$.getJSON() method. The retrieved data was processed and dynamically displayed in an HTML table using JavaScript. The experiment successfully demonstrated JSON handling, asynchronous data loading, jQuery, and dynamic DOM manipulation. Thus, the objectives of the experiment were achieved successfully.